

String Palindrome

```
String str = "madam";
boolean isPalindrome = IntStream.range(0, str.length() / 2)
    .allMatch(i -> str.charAt(i) == str.charAt(str.length() - 1 - i));
System.out.println(isPalindrome ? "Palindrome" : "Not");
```

Number palindrome

```
int num = 1271;
String str = String.valueOf(num);
boolean isPalindrome = IntStream.range(0, str.length() / 2)
    .allMatch(i -> str.charAt(i) == str.charAt(str.length() - 1 - i));
System.out.println(isPalindrome ? "Palindrome" : "Not");
```

String reverse

```
String name = "ankur";
String reversedString = IntStream.range(0, name.length()).mapToObj(i -> name.charAt(str.length() - i))
    .map(String::valueOf).collect(Collectors.joining());
System.out.println(reversedString);
```

String anagram

```
String str1 = "listen", str2 = "silent";

boolean isAnagram = str1
    .chars()
    .mapToObj(item -> String.valueOf((char) item))
```

```
        .sorted()
        .collect(Collectors.joining())
        .equals(str2
        .chars()
        .mapToObj(item1 -> String.valueOf((char) item1))
        .sorted()
        .collect(Collectors.joining())));
System.out.println(isAnagram ? "Anagram" : "Not");
```

Number Anagram

```
public class NumberAnagram {
    public static boolean isAnagram(int num1, int num2) {
        // Convert numbers to char arrays
        char[] arr1 = String.valueOf(num1).toCharArray();
        char[] arr2 = String.valueOf(num2).toCharArray();

        // Sort using streams
        //Option 1
        String sorted1 = Arrays.stream(String.valueOf(num1).split(""))
            .sorted()
            .collect(StringBuilder::new, StringBuilder::append, StringBuilder::append).toString();

        String sorted2 = Arrays.stream(String.valueOf(num2).split(""))
            .sorted()
            .collect(StringBuilder::new, StringBuilder::append, StringBuilder::append).toString();

        //Option 2
        String sorted1 = Arrays.stream(String.valueOf(num1).split(""))
            .sorted().collect(Collectors.joining());
        String sorted2 = Arrays.stream(String.valueOf(num2).split(""))
            .sorted().collect(Collectors.joining());
        return sorted1.equals(sorted2);
    }

    public static void main(String[] args) {
        int num1 = 12345;
        int num2 = 54321;
        int num3 = 12435;
        int num4 = 12344;
        System.out.println(num1 + " & " + num2 + " are anagrams? " + isAnagram(num1, num2));
    }
}
```

```
System.out.println(num1 + " & " + num3 + " are anagrams? " + isAnagram(num1, num3));  
System.out.println(num1 + " & " + num4 + " are anagrams? " + isAnagram(num1, num4));  
}  
}
```

Sum of all elements in list or array

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9);  
int sum = IntStream.range(0, nums.size()).sum();  
System.out.println(sum);
```

Length of array or list

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9);  
long arrayLength = nums.stream().count();  
System.out.println(arrayLength);
```

Length of array or list by incrementing count

```
AtomicInteger count = new AtomicInteger(0);  
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9);  
nums.stream().forEach(item -> {  
    count.addAndGet(1);  
});  
System.out.println(count.get());
```

Find sum, count, max, min, average of all elements in list or array

```
List<Integer> nums = Arrays.asList(1, 1, 2, 2);  
IntSummaryStatistics sum = nums.stream().mapToInt(item -> item.intValue()).summaryStatistics();  
System.out.println("Average: " + sum.getAverage());  
System.out.println("Max: " + sum.getMax());  
System.out.println("Min: " + sum.getMin());  
System.out.println("Sum: " + sum.getSum());
```

```
System.out.println("Count: " + sum.getCount());
```